

Software Requirements Specification

For

CampusCompute: A Private Academic Cloud for Utilizing Idle Laboratory Resources

October 2026

Prepared by

S.No	Specialization	SAP ID	Name
1	FullStack AI B-1	500120595	Kuldeep Chaudhary
2	CCVT B-1	500119484	Rishu Raj
3	CCVT B-2	500125960	Daksh Mehrotra
4	CCVT B-5	500122440	Krishna Bisht

Project Mentor: Dr. Mohd Hanief Wani



Department of Informatics
School of Computer Science
UNIVERSITY OF PETROLEUM & ENERGY STUDIES,
DEHRADUN- 248007, Uttarakhand

Contents

Revision History	2
1 INTRODUCTION	3
1.1 Purpose of the Project	3
1.2 Target Beneficiary	3
1.3 Project Scope	3
1.4 References	4
2 PROJECT DESCRIPTION	4
2.1 Reference Algorithm	4
2.2 Characteristic of Data	6
2.3 SWOT Analysis	7
2.4 Project Features	7
2.5 User Classes and Characteristics	8
2.6 Design and Implementation Constraints	9
2.7 Design Diagrams	11
2.8 Assumption and Dependencies	12
3 SYSTEM REQUIREMENTS	14
3.1 User Interface	14
3.2 Software Interface	16
3.3 Database Interface	19
3.4 Protocols	21
4 NON-FUNCTIONAL REQUIREMENTS	22
4.1 Performance Requirements	22
4.2 Security Requirements	24
4.3 Software Quality Attributes	25
5 OTHER REQUIREMENTS	27
Appendix A: Glossary	28
Appendix B: Analysis Model	29
Appendix C: Issues List	30

REVISION HISTORY

Date	Change	Reason for Changes	Mentor Signature
August 15, 2026	Initial draft	Project scope definition and requirement gathering	
September 5, 2026	Added system architecture	Technical design and component specification	
September 18, 2026	Updated diagrams	Added all UML diagrams per template requirements	
September 28, 2026	Document finalization	Final version after mentor feedback and corrections	

1 INTRODUCTION

1.1 Purpose of the Project

CampusCompute addresses underutilized computing resources in educational institutions (60-70% idle time) while providing students free access to high-performance infrastructure. Most college labs remain idle during non-operating hours, weekends, and holidays despite significant capital investment (Rs. 1-5 Crore). Meanwhile, students face prohibitive cloud service costs (Rs. 2000-5000/month).

CampusCompute transforms idle lab computers into a private cloud, enabling on-demand isolated containerized environments accessible 24/7 from anywhere. At UPES, 250 lab computers (Rs. 1.25 Crore) with 1500 CPU cores operate at only 20% utilization. CampusCompute increases this to 60-70%, multiplying institutional computing capacity without additional hardware.

1.2 Target Beneficiary

Students: CS/Data Science/AI-ML students needing high-performance computing for projects; economically disadvantaged students unable to afford expensive hardware or cloud subscriptions; remote learners requiring lab access outside campus hours.

Educational Institutions: IT departments maximizing ROI on lab infrastructure; lab administrators managing 100-500 computers; academic departments with cloud budget constraints.

Faculty: Course instructors assigning computationally intensive projects; researchers needing distributed computing; lab coordinators monitoring resource usage.

1.3 Project Scope

CampusCompute applies to educational institutions with existing lab infrastructure, enabling resource pooling, on-demand container provisioning, and secure multi-tenant access.

Benefits: Eliminate Rs. 5-10 lakhs annual cloud costs; increase utilization from 20% to 60-70%; 24/7 remote access; support 500+ devices and 2000+ containers; isolated secure environments.

Objectives:

1. Deploy distributed container orchestration across campus labs
2. Implement intelligent scheduling for optimal resource allocation
3. Provide web interface for container management and terminal access
4. Establish secure WebSocket communication between backend and agents
5. Support multi-organization architecture with quota management

Goals: Container creation in 5-10 seconds; support 100+ concurrent devices with 500+ containers; 99%+ uptime; reduce student cloud expenses by 90-95%; one-click agent deployment.

Deliverables: Backend (Spring Boot with REST/WebSocket), Frontend (React portal), Python agent, Windows installer, PostgreSQL schema, documentation (installation, API, user manuals), architecture diagrams.

1.4 References

1. Docker Documentation - Container Platform Architecture. <https://docs.docker.com/>
2. Spring Boot Framework Documentation. <https://spring.io/projects/spring-boot>
3. WebSocket Protocol (RFC 6455). <https://datatracker.ietf.org/doc/html/rfc6455>
4. PostgreSQL Database Documentation. <https://www.postgresql.org/docs/>
5. React Frontend Framework. <https://react.dev/>
6. xterm.js - Terminal Emulator for Web. <https://xtermjs.org/>
7. IEEE Standard for Software Requirements Specifications (IEEE 830-1998)
8. Project Documentation: PROJECT_DOCUMENTATION.md (Internal)

2 PROJECT DESCRIPTION

2.1 Reference Algorithm

Primary Algorithm: Adaptive Device Scoring and Selection

CampusCompute implements a multi-factor scoring algorithm for intelligent device selection when allocating containers. This algorithm ensures optimal resource distribution, load balancing, and high availability.

Algorithm Steps:

Function: `selectBestDevice(requestedCPU, requestedRAM, requestedDisk)`

Input: Container resource requirements

Output: Selected Device ID

1. Fetch all ONLINE devices from database
2. For each device:
 - a. Calculate availableResources:
 - `availableCPU = totalCPU - allocatedCPU`
 - `availableRAM = totalRAM - allocatedRAM`
 - `availableDisk = totalDisk - allocatedDisk`
 - b. Check if device can fulfill request:
IF (`availableCPU >= requestedCPU` AND
 `availableRAM >= requestedRAM` AND
 `availableDisk >= requestedDisk`)
THEN device is candidate
ELSE skip device

c. Calculate device score (0-100):

$$\text{score} = (w1 * \text{cpuScore}) + (w2 * \text{ramScore}) + \\ (w3 * \text{diskScore}) + (w4 * \text{containerScore}) + \\ (w5 * \text{healthScore})$$

Where:

- $\text{cpuScore} = (\text{availableCPU} / \text{totalCPU}) * 100$
- $\text{ramScore} = (\text{availableRAM} / \text{totalRAM}) * 100$
- $\text{diskScore} = (\text{availableDisk} / \text{totalDisk}) * 100$
- $\text{containerScore} = (1 - \text{activeContainers}/\text{maxContainers}) * 100$
- $\text{healthScore} = \text{deviceHealthStatus}$ (100=healthy, 50=warning, 0=error)

Weights (configurable):

- $w1 = 0.30$ (CPU weight)
- $w2 = 0.30$ (RAM weight)
- $w3 = 0.15$ (Disk weight)
- $w4 = 0.15$ (Container count weight)
- $w5 = 0.10$ (Health weight)

3. Sort candidates by score (descending)
4. Return device with highest score
5. If no candidate found, throw `NoDeviceAvailableException`

Example Calculation:

Device Specs: 8 CPU cores, 16GB RAM, 500GB disk, 3 active containers

Request: 2 CPU, 4GB RAM, 50GB disk

Available: 6 CPU, 12GB RAM, 450GB disk

$\text{cpuScore} = (6/8) * 100 = 75.0$

$\text{ramScore} = (12/16) * 100 = 75.0$

$\text{diskScore} = (450/500) * 100 = 90.0$

$\text{containerScore} = (1 - 3/10) * 100 = 70.0$

$\text{healthScore} = 100$ (healthy device)

$$\begin{aligned} \text{Final Score} &= (0.30*75) + (0.30*75) + (0.15*90) + (0.15*70) + (0.10*100) \\ &= 22.5 + 22.5 + 13.5 + 10.5 + 10 \\ &= 79.0 \end{aligned}$$

Data Structures Used:

1. **Priority Queue/List:** For sorting devices by computed scores

2. **Hash Map:** For storing device metrics (deviceId to metrics)
3. **Database Indices:** B-tree indices on device status, organization_id for fast queries
4. **WebSocket Connection Pool:** HashMap mapping deviceId to WebSocket session
5. **Container State Machine:** Finite State Automaton (FSA) managing lifecycle transitions

2.2 Characteristic of Data

Primary Data Sources:

1. System Metrics (Real-time):

- Source: Agent monitoring (psutil library)
- Frequency: Every 60 seconds
- Metrics: CPU usage (%), RAM usage (bytes), disk usage (bytes), network I/O
- Processing: Time-series aggregation, moving averages

2. Container Metadata:

- Source: Docker Engine API via docker-py
- Data: Container ID, status, image, ports, resource limits, creation timestamp
- Storage: PostgreSQL relational database

3. User Activity Logs:

- Source: Backend application logs
- Data: User actions, API calls, authentication events, container operations
- Storage: Structured logs (JSON format)
- Processing: Log aggregation for analytics

4. Device Inventory:

- Source: Agent enrollment process
- Data: Hostname, CPU specs, RAM capacity, disk space, operating system
- Collection Method: One-time during installation, updated on reconnection

Secondary Data Sources:

1. Docker Hub public images metadata
2. Configuration files (YAML format)
3. Enrollment tokens database

Data Processing Methods:

- **Normalization:** Resource metrics normalized to 0-100 scale for scoring
- **Aggregation:** Real-time metrics aggregated per device and organization
- **Filtering:** SQL queries with WHERE clauses filtering by status, organization
- **Validation:** Input validation using Bean Validation (JSR-380)

Strengths	Weaknesses
• Zero cost for students (free cloud alternative) • Utilizes existing infrastructure (no new hardware) • 93% cost savings vs AWS/Azure • On-premise data (privacy & compliance) • Low latency (~10ms LAN vs 50-200ms WAN) • Simple user experience (1-click container creation) • Multi-organization support (scalable architecture)	• Limited to campus lab capacity • Requires Docker on each agent machine • Network dependency (WebSocket connectivity) • Windows-centric installer (Linux manual setup) • Initial setup complexity for IT administrators • Not suitable for GPU-intensive workloads (current version)
Opportunities	Threats
• Expand to 500+ institutions nationwide • Add GPU support for AI/ML workloads • Integration with LMS (Moodle, Canvas) • Marketplace for container images • Federation (share resources between universities) • White-label SaaS offering • Government grants for educational technology	• Cloud providers offering student credits (AWS Educate) • Security vulnerabilities in container escape • Resistance from IT departments (change management) • Docker licensing changes • Network firewall restrictions • Competing open-source solutions (OpenStack)

Table 1: SWOT Analysis of CampusCompute

2.3 SWOT Analysis

2.4 Project Features

Major Features:

1. Container Lifecycle Management

- Create containers with custom CPU, RAM, disk allocations
- AWS-style state transitions (PENDING to CREATING to RUNNING to STOPPING to STOPPED to RESTARTING to DELETED)
- Real-time status updates via WebSocket
- Automatic container expiration and cleanup
- Support for multiple Docker images (Ubuntu, Alpine, Python, Node.js)

2. Intelligent Resource Scheduling

- Multi-factor device scoring algorithm
- Load balancing across devices
- Quota enforcement per user and organization
- Resource limit validation before allocation
- Priority-based scheduling (future enhancement)

3. Web-Based Terminal Access

- Browser-based terminal using xterm.js
- WebSocket-based real-time communication
- Full shell access to containers
- Support for command history, scrollbar buffer
- Secure authenticated sessions

4. Multi-Tenancy and Role-Based Access Control

- Organization-level isolation
- Three roles: STUDENT, FACULTY, ADMIN
- JWT-based authentication
- Resource quotas per organization
- Lab-level access control

5. Real-Time Monitoring and Analytics

- Device health monitoring (CPU, RAM, disk, network)
- Container metrics collection
- Heartbeat mechanism (30-second intervals)
- Automatic device offline detection
- Admin dashboard with live statistics

6. One-Click Agent Deployment

- Windows GUI installer
- Automatic Docker Desktop installation
- Automatic Python environment setup
- Windows service registration
- Enrollment token-based authentication
- Progress tracking with detailed logging

7. Device and Lab Management

- Register and manage multiple labs
- Generate enrollment tokens for device onboarding
- View device status, specifications, container counts
- Update device metadata
- Remove/decommission devices

8. User Management

- Bulk student registration via CSV upload
- User creation with roles and quotas
- Password reset and first-time login flow
- User activity tracking
- Organization-specific user isolation

2.5 User Classes and Characteristics

User Class 1: Student

Characteristics: Primary users (80-90%); basic to intermediate proficiency; age 18-25; access 3-5 times/week for 1-4 hours.

Use Cases: Login, create containers with specific resources, access web terminal, run code/compile programs, stop/restart containers, view container list.

Requirements: Web browser, internet, basic Linux knowledge.

User Class 2: Faculty

Characteristics: Instructors and researchers; intermediate to advanced proficiency; access 2-3 times/week.

Use Cases: View student usage, create demo containers, reserve labs, monitor utilization, generate reports.

User Class 3: Administrator

Characteristics: IT staff; advanced proficiency; daily access for configuration and maintenance.

Use Cases: Manage organizations/labs, generate enrollment tokens, monitor devices/containers, manage users, configure quotas, view analytics, troubleshoot connectivity, perform bulk operations.

Requirements: Backend access, Docker/networking/database knowledge, lab computer access.

User Class 4: Agent (System User)

Characteristics: Automated software component running as Windows/Linux service with persistent WebSocket connection.

Operations: Maintain WebSocket connection, report metrics (60s), execute container commands, send heartbeat (30s), handle reconnection.

2.6 Design and Implementation Constraints

Hardware: Lab computers minimum 4 CPU, 8GB RAM, 100GB disk; backend minimum 4 CPU, 8GB RAM (16GB recommended), 100GB SSD; 100 Mbps LAN; Docker compatibility required.

Software:

- **Operating System:**

- Backend: Windows/Linux/macOS
- Agent: Windows 10/11 (64-bit) or Linux (Ubuntu 20.04+)
- Client: Any OS with modern web browser

- **Mandatory Dependencies:**

- Java 17+ (backend)
- PostgreSQL 15+ (database)
- Docker Desktop 24+ or Docker Engine 24+ (agents)
- Python 3.11+ (agent)
- Node.js 18+ (frontend build)

- **Browser Compatibility:** Chrome 90+, Firefox 88+, Edge 90+, Safari 14+

Technology Constraints:

- **Framework:** Spring Boot 3.2.x (requires Java 17+)
- **WebSocket Protocol:** RFC 6455 compliance required
- **Container Runtime:** Docker only (no Podman/containerd support currently)
- **Database:** PostgreSQL (not compatible with MySQL/Oracle without schema changes)

Timing Requirements:

- Container creation: 5-10 seconds (target), 15 seconds (maximum acceptable)
- API response time: ≤ 200 ms for 95% of requests
- WebSocket message latency: ≤ 50 ms
- Heartbeat interval: 30 seconds
- Metrics collection: 60 seconds
- Container expiration check: 5 minutes

Memory Requirements:

- Backend JVM: 2GB heap (minimum), 4GB (recommended)
- Agent process: 100MB (idle), 200MB (active)
- Frontend build: 2GB RAM during build
- Database: 4GB (100 devices), 8GB (500 devices)

Security Constraints:

- JWT tokens expire after 24 hours
- Passwords must be hashed using BCrypt
- WebSocket connections must use WSS (WebSocket Secure) in production
- Enrollment tokens valid for 24 hours, single-use only
- Containers run without privileged access
- Network isolation between containers

Interfaces to Other Applications:

- Docker Engine API (via REST and Unix socket)
- PostgreSQL database (via JDBC)
- Redis cache (optional, via Jedis client)
- Docker Hub (for pulling container images)

Communication Protocols:

- HTTP/HTTPS (REST APIs)
- WebSocket/WSS (real-time communication)
- TCP (database connections)
- Docker API over Unix socket or named pipe

Language Requirements:

- Backend: Java 17 (Oracle JDK or OpenJDK)
- Frontend: JavaScript ES6+, JSX
- Agent: Python 3.11+ (async/await syntax required)
- Database: SQL (PostgreSQL dialect)

Design Conventions:

- RESTful API design principles
- JSON for data interchange
- Camel case for Java variables, snake_case for database columns
- Semantic versioning for releases (MAJOR.MINOR.PATCH)

Programming Standards:

- Java: Google Java Style Guide
- JavaScript: Airbnb JavaScript Style Guide
- Python: PEP 8 style guide
- Code documentation: Javadoc (Java), JSDoc (JavaScript), docstrings (Python)

2.7 Design Diagrams

UPES Template: 8 diagrams for Major Projects (6 standard + 2 additional). Status: 6/8 included.

2.7.1 Use Case Diagram (Level 2)

Students create and manage containers. Faculty monitor activity. Administrators manage organizations, users, devices. Agents execute commands and report metrics.

2.7.2 Class Diagram

Organization contains Users and Labs. Labs have Devices which run Containers. Users create Containers.

2.7.3 Activity Diagram & 2.7.4 Sequence Diagram

These requirements are fulfilled by Collaboration Diagram (Section 2.7.7) showing object interactions and temporal sequence through numbered messages (1-15).

2.7.5 Data Flow Diagram (Level 0)

CampusCompute as single process with external entities: Student, Faculty, Admin, Lab Computer Agent.

2.7.6 State Diagram

Export needed: diagrams/2_container_lifecycle.drawio to ContainerLifecycle.png

Container lifecycle: 8 states - PENDING, CREATING, RUNNING, STOPPING, STOPPED, RESTARTING, FAILED, DELETED. AWS-style intermediate states ensure UI reflects actual status during transitions.

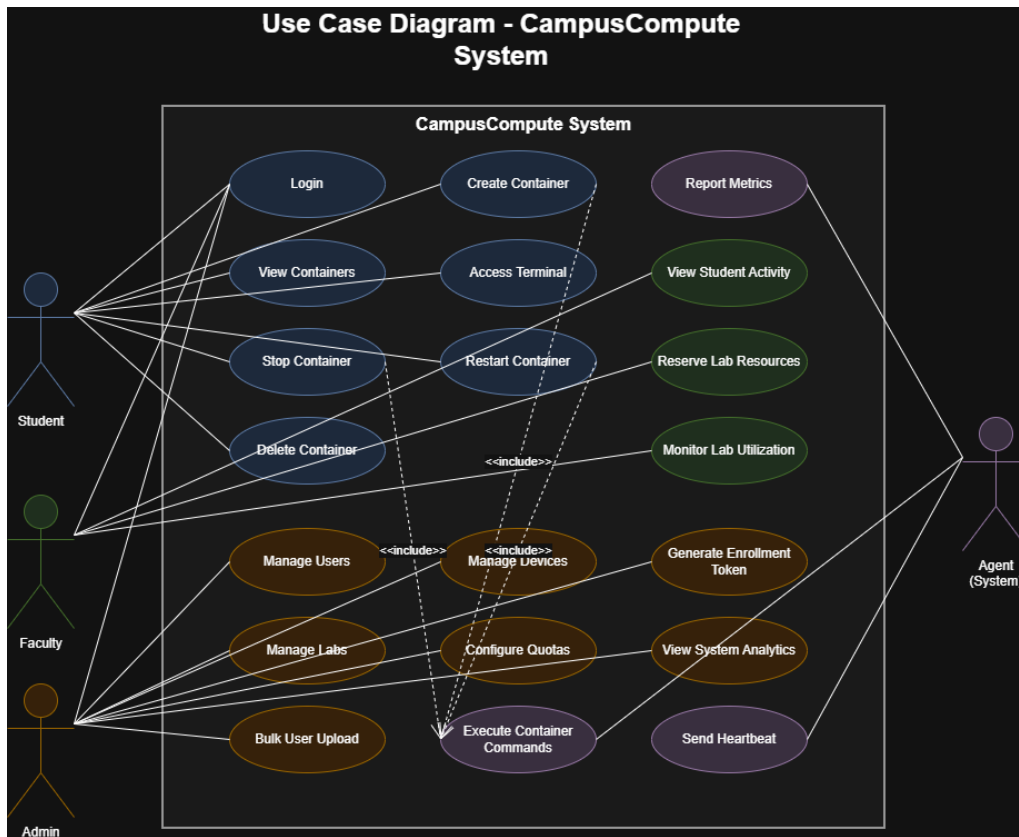


Figure 1: Use Case Diagram - 4 actors, 20 use cases

2.7.7 Collaboration Diagram

Complete sequence: (1) Student request, (2-3) Frontend/Controller, (4-7) Scheduler/Database, (8-9) Save container, (10-11) WebSocket command, (12-14) Agent/Docker execution, (15) Confirmation.

2.7.8 Deployment Diagram

Export needed: diagrams/6_deployment_architecture.drawio to DeploymentArchitecture.png

Production infrastructure: 3 zones - DMZ (frontend/backend, WebSocket gateway, Redis, monitoring), Internal Network (PostgreSQL primary/replica, backups, NAS), Lab Network (150+ devices, Python Agent, Docker Engine across 15 labs).

Cost Analysis: On-Premise Rs. 8 lakhs (one-time) + Rs. 80k/year vs AWS Rs. 12 lakhs/year. Savings: Rs. 11.2 lakhs/year (93%).

2.7.9 System Architecture (Supplementary)

The system follows a 3-tier architecture: Frontend layer (React SPA with terminal emulator), Backend layer (Spring Boot with REST APIs and WebSocket handlers), and Agent layer (Python agents on lab computers with Docker integration). Communication uses HTTP/REST for management operations and WebSocket for real-time terminal access and container commands.

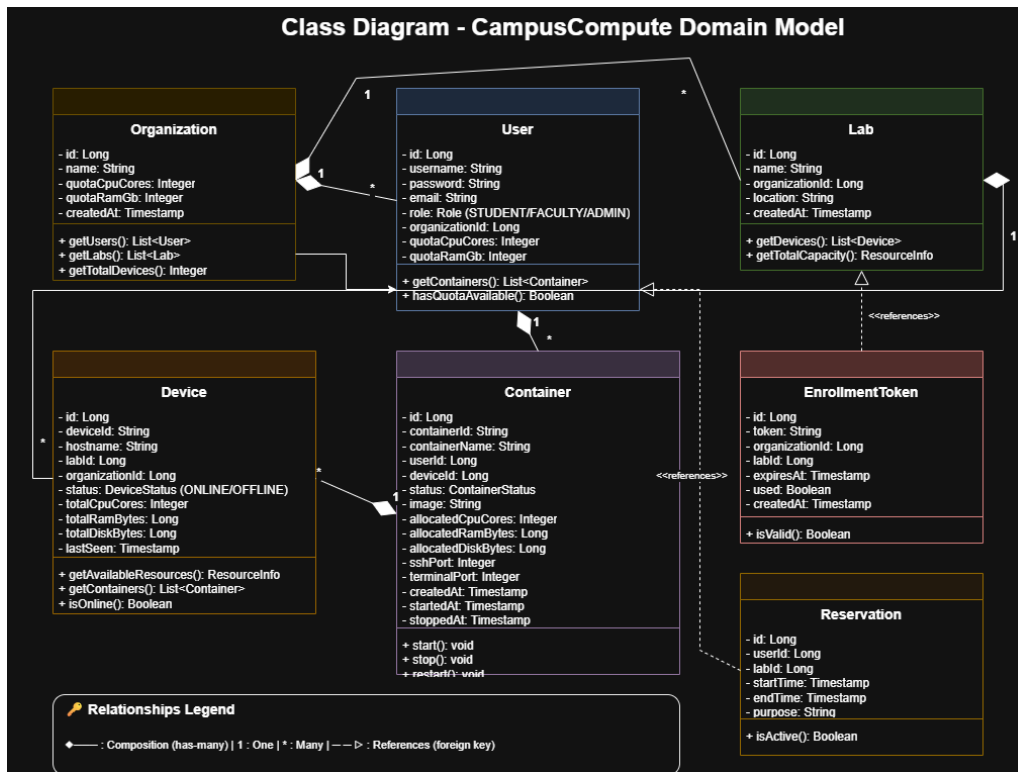


Figure 2: Domain Class Diagram - 7 entities with relationships

2.8 Assumption and Dependencies

Assumptions:

1. Lab computers remain powered on during operating hours (8 AM - 10 PM)
2. Stable network connectivity between backend and agent machines
3. Institutional firewall allows WebSocket connections on port 8081
4. Docker Desktop licenses are available for Windows lab computers
5. Students have basic Linux command-line knowledge
6. IT administrators can perform one-time agent installation on lab computers
7. Backend server has static IP address or DNS hostname
8. Lab computers are not heavily used for other purposes during idle hours
9. Students will not attempt container escape or malicious activities
10. Institutional network has sufficient bandwidth for concurrent terminal sessions

Dependencies:

External Dependencies:

- **Docker Inc.:** Docker Desktop and Docker Engine availability, licensing terms
- **Docker Hub:** Public image repository accessibility
- **Java/Oracle:** Java 17 runtime availability
- **PostgreSQL:** Database engine stability and updates
- **Python Software Foundation:** Python 3.11+ interpreter
- **Spring Framework:** Spring Boot updates and security patches

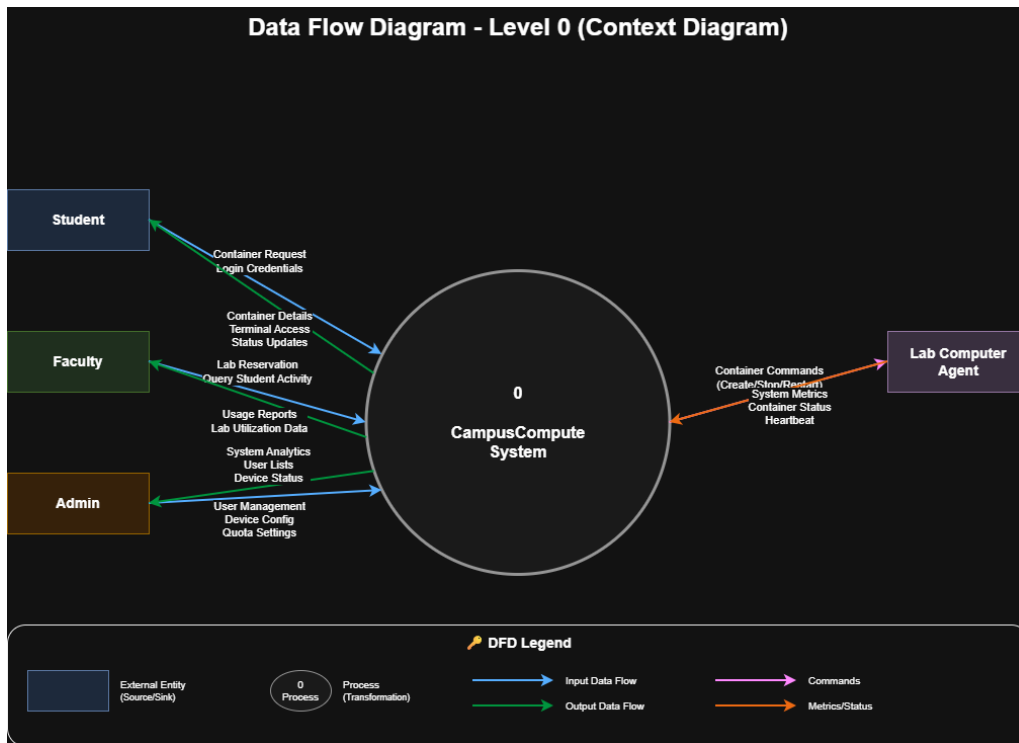


Figure 3: Context-level DFD - 4 external entities, 8 data flows

Library Dependencies:

- Spring Boot Starter Web, WebSocket, Security, Data JPA
- React, Material-UI, Redux Toolkit
- Python: websockets, docker-py, psutil, pyyaml, pywin32
- xterm.js for terminal emulation
- PostgreSQL JDBC driver

Infrastructure Dependencies:

- Backend server with public IP or campus network accessibility
- PostgreSQL database server (can be containerized)
- Domain name or IP address for frontend hosting
- SSL/TLS certificates for HTTPS in production

Organizational Dependencies:

- IT department approval for agent installation
- Network administrator cooperation for firewall configuration
- User management system integration (optional)
- Institutional policies on student resource usage

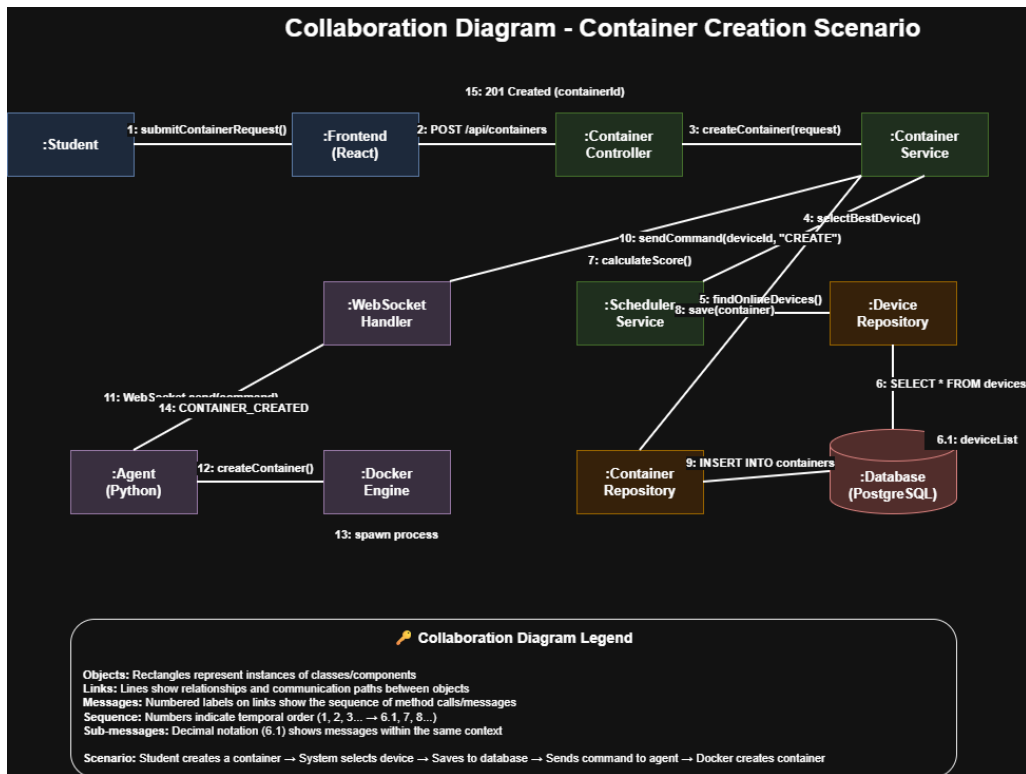


Figure 4: Collaboration Diagram - Container creation with 15 messages

3 SYSTEM REQUIREMENTS

3.1 User Interface

3.1.1 Student Portal Interface

Components:

- **Login Page:** Username/password input, "Remember Me" checkbox, "Forgot Password" link
- **Dashboard:** Container list, resource usage summary, quick create button
- **Create Container Dialog:** Dropdowns for CPU (1-8 cores), RAM (1-16GB), Disk (10-100GB), Image selection (Ubuntu, Alpine, Python, Node.js)
- **Container Card:** Status badge (color-coded), container name, resource allocations, action buttons (Start, Stop, Restart, Delete, Open Terminal)
- **Web Terminal:** Full-screen terminal emulator with xterm.js, resize support, scrollbar buffer

Design Requirements:

- Responsive design (desktop, tablet, mobile)
- Material Design principles (Material-UI components)
- Loading states with spinners
- Error messages with actionable feedback
- Success notifications (snackbars)

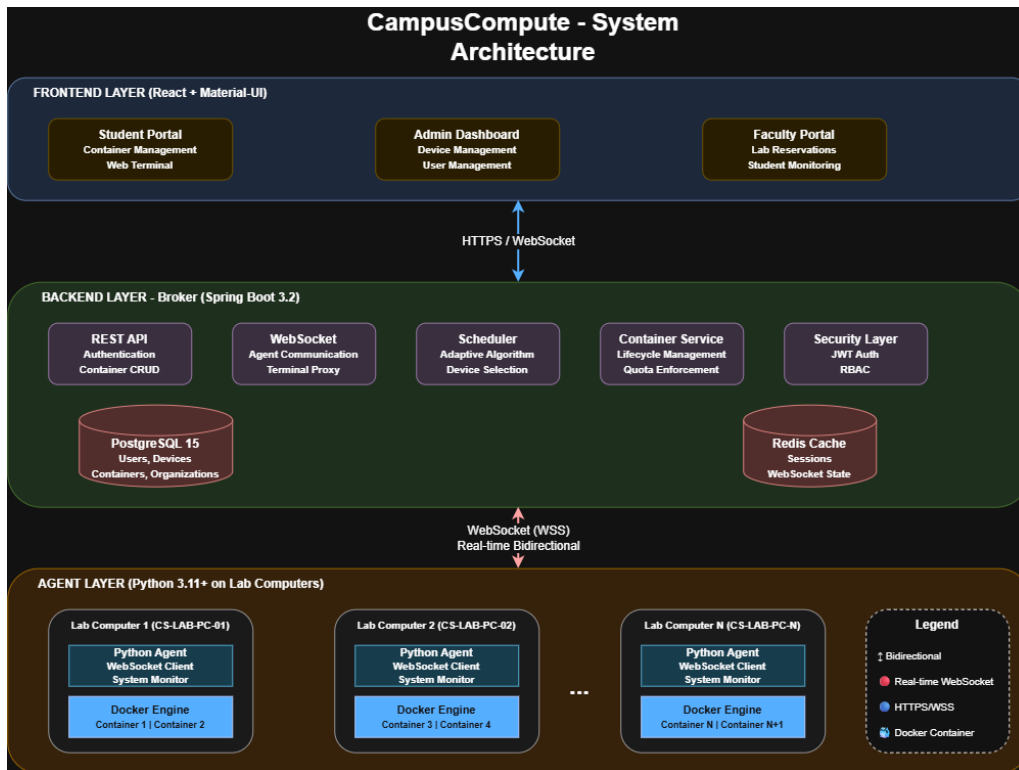


Figure 5: System Architecture - 3-layer design with WebSocket connections

- Real-time status updates via WebSocket

3.1.2 Admin Dashboard Interface

Components:

- **Overview Page:** Total devices, active containers, online/offline devices, organization statistics
- **Device Management:** Table with device name, status, CPU/RAM/Disk specs, container count, last seen timestamp, actions (View Details, Remove)
- **User Management:** Table with username, role, organization, active containers, quota usage, actions (Edit, Delete, Reset Password)
- **Lab Management:** List of labs with name, device count, total resources, actions (Edit, Delete)
- **Enrollment Tokens:** Generate token dialog with lab selection, expiration time, view existing tokens
- **Bulk Upload:** CSV file upload for student registration with validation and error reporting

Design Requirements:

- Data tables with sorting, filtering, pagination
- Charts and graphs for analytics (CPU/RAM utilization over time)
- Modal dialogs for forms
- Confirmation dialogs for destructive actions
- Export functionality (CSV, PDF) for reports

3.1.3 Agent Installer Interface

Components:

- **Welcome Screen:** Project logo, description, "Next" button
- **Enrollment Token Input:** Text field with validation
- **Installation Progress:** Progress bar (0-100%), current step description, detailed log output
- **Completion Screen:** Success message, "Finish" button, option to view logs

Design Requirements:

- Simple wizard-style interface
- Clear step-by-step instructions
- Error handling with retry options
- Windows-native look and feel (tkinter)

3.2 Software Interface

3.2.1 Frontend - Backend Interface

Protocol: HTTP/HTTPS (REST) and WebSocket/WSS

REST API Endpoints:

POST	/api/auth/login	- User authentication
POST	/api/auth/logout	- Logout
GET	/api/containers	- List user containers
POST	/api/containers	- Create container
DELETE	/api/containers/{id}	- Delete container
POST	/api/containers/{id}/stop	- Stop container
POST	/api/containers/{id}/restart	- Restart container
GET	/api/devices	- List devices (admin)
GET	/api/users	- List users (admin)
POST	/api/users	- Create user
POST	/api/labs	- Create lab
GET	/api/enrollment-tokens	- List tokens
POST	/api/enrollment-tokens/generate	- Generate token

WebSocket Endpoints:

/ws/terminal/{containerId}	- Terminal session
/ws/status	- Real-time status updates

Data Format: JSON

Authentication: JWT token in Authorization header: "Bearer {token}"

Error Responses: Standard HTTP status codes with JSON error objects

3.2.2 Backend - Agent Interface

Protocol: WebSocket (bidirectional persistent connection)

Connection URL:

`ws://backend:8081/ws/agent?enrollmentToken=xxx&deviceId=yyy`

Message Types (Backend to Agent):

CREATE_CONTAINER	- Create new container
STOP_CONTAINER	- Stop running container
RESTART_CONTAINER	- Restart container
DELETE_CONTAINER	- Remove container
PING	- Heartbeat check

Message Types (Agent to Backend):

HEARTBEAT	- Keep-alive signal
CONTAINER_CREATED	- Container successfully created
CONTAINER_STOPPED	- Container stopped
CONTAINER_RESTARTED	- Container restarted
CONTAINER_DELETED	- Container deleted
CONTAINER_FAILED	- Operation failed
METRICS	- System resource metrics

Message Format:

```
{
  "type": "CREATE_CONTAINER",
  "deviceId": 123,
  "organizationId": 1,
  "requestId": "6",
  "timestamp": "2026-10-06T10:30:00Z",
  "payload": {
    "container_name": "container-user-1234",
    "image": "ubuntu:latest",
    "cpu_cores": 2,
    "ram_bytes": 4294967296,
    "disk_bytes": 53687091200
  }
}
```

Communication Security:

- Enrollment token authentication on first connection

- Device ID-based authentication on subsequent connections
- Message integrity via JSON schema validation

3.2.3 Agent - Docker Interface

Protocol: Docker Engine API via docker-py (Python SDK)

Connection Methods:

- Windows: Named pipe (npipe:///./pipe/docker_engine)
- Linux: Unix socket (unix:///var/run/docker.sock)

Docker SDK Operations:

```
client.containers.run()      - Create and start container
container.stop()            - Stop container
container.restart()         - Restart container
container.remove()          - Delete container
client.containers.list()    - List containers
client.info()               - Get Docker info
```

Container Configuration:

```
{
  "image": "ubuntu:latest",
  "name": "container-user-1234",
  "detach": True,
  "tty": True,
  "stdin_open": True,
  "cpu_count": 2,
  "mem_limit": "4g",
  "storage_opt": {"size": "50g"},
  "network_mode": "bridge"
}
```

3.3 Database Interface

Database Management System: PostgreSQL 15+

Connection Parameters:

```
Driver: org.postgresql.Driver
URL: jdbc:postgresql://localhost:5432/campuscompute
Username: campuscompute
Password: <configured>
Connection Pool: HikariCP (default in Spring Boot)
Max Pool Size: 20 connections
```

Key Tables:

users:

- id (BIGSERIAL PRIMARY KEY)
- username (VARCHAR UNIQUE)
- password (VARCHAR, BCrypt hashed)
- email (VARCHAR)
- role (VARCHAR: STUDENT, FACULTY, ADMIN)
- organization_id (BIGINT FK)
- quota_cpu_cores (INT)
- quota_ram_gb (INT)
- first_time_login (BOOLEAN)

devices:

- id (BIGSERIAL PRIMARY KEY)
- device_id (VARCHAR UNIQUE)
- hostname (VARCHAR)
- lab_id (BIGINT FK)
- organization_id (BIGINT FK)
- status (VARCHAR: ONLINE, OFFLINE)
- total_cpu_cores (INT)
- total_ram_bytes (BIGINT)
- total_disk_bytes (BIGINT)
- last_seen (TIMESTAMP)

containers:

- id (BIGSERIAL PRIMARY KEY)
- container_id (VARCHAR UNIQUE)
- container_name (VARCHAR)
- user_id (BIGINT FK)
- device_id (BIGINT FK)
- status (VARCHAR: PENDING, CREATING, RUNNING, STOPPING, STOPPED, RESTARTING, FAILED, DELETED)
- image (VARCHAR)
- allocated_cpu_cores (INT)
- allocated_ram_bytes (BIGINT)
- allocated_disk_bytes (BIGINT)
- created_at (TIMESTAMP)
- started_at (TIMESTAMP)
- stopped_at (TIMESTAMP)
- expires_at (TIMESTAMP)

organizations:

- id (BIGSERIAL PRIMARY KEY)
- name (VARCHAR UNIQUE)
- quota_cpu_cores (INT)
- quota_ram_gb (INT)
- created_at (TIMESTAMP)

labs:

- id (BIGSERIAL PRIMARY KEY)
- name (VARCHAR)
- organization_id (BIGINT FK)
- location (VARCHAR)
- created_at (TIMESTAMP)

enrollment_tokens:

- id (BIGSERIAL PRIMARY KEY)
- token (VARCHAR UNIQUE)
- organization_id (BIGINT FK)
- lab_id (BIGINT FK)
- expires_at (TIMESTAMP)
- used (BOOLEAN)
- created_at (TIMESTAMP)

Indices:

```
CREATE INDEX idx_containers_user ON containers(user_id);
CREATE INDEX idx_containers_device ON containers(device_id);
CREATE INDEX idx_containers_status ON containers(status);
CREATE INDEX idx_devices_status ON devices(status);
CREATE INDEX idx_devices_org ON devices(organization_id);
CREATE INDEX idx_users_org ON users(organization_id);
```

Constraints:

```
ALTER TABLE containers ADD CONSTRAINT containers_status_check
    CHECK (status IN ('PENDING', 'CREATING', 'RUNNING', 'STOPPING',
                      'STOPPED', 'RESTARTING', 'FAILED', 'DELETED'));

ALTER TABLE devices ADD CONSTRAINT devices_status_check
    CHECK (status IN ('ONLINE', 'OFFLINE'));

ALTER TABLE users ADD CONSTRAINT users_role_check
    CHECK (role IN ('STUDENT', 'FACULTY', 'ADMIN'));
```

3.4 Protocols

3.4.1 WebSocket Protocol

Standard: RFC 6455 (WebSocket Protocol)

Upgrade Request:

```
GET /ws/agent?enrollmentToken=abc123 HTTP/1.1
Host: backend:8081
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: <base64-encoded-key>
Sec-WebSocket-Version: 13
```

Upgrade Response:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: <hash>
```

Frame Format: Binary or text frames with JSON payload

Ping/Pong: Heartbeat every 30 seconds

Closure: Graceful close with status code 1000 (Normal Closure)

3.4.2 HTTP/HTTPS Protocol

Standard: HTTP/1.1 (RFC 7230-7235)

Methods Used: GET, POST, PUT, DELETE

Headers:

```
Authorization: Bearer <JWT-token>
Content-Type: application/json
Accept: application/json
```

Status Codes:

200 OK	- Successful request
201 Created	- Resource created
400 Bad Request	- Invalid input
401 Unauthorized	- Authentication required
403 Forbidden	- Insufficient permissions
404 Not Found	- Resource not found
500 Internal Server Error	- Server error

3.4.3 Security Protocols

Authentication: JWT (JSON Web Tokens) using HMAC-SHA256

Token Structure:

Header: {"alg": "HS256", "typ": "JWT"}

Payload: {"sub": "username", "role": "STUDENT",
 "organizationId": 1, "exp": 1696598400}

Signature: HMACSHA256(base64(header)+"."+base64(payload), secret)

Password Hashing: BCrypt with cost factor 10

HTTPS: TLS 1.2 or 1.3 (production deployment)

3.4.4 Communication Security

Encryption:

- Production: WSS (WebSocket Secure) with TLS 1.3
- Development: WS (unencrypted) for testing
- HTTPS for REST APIs in production

Data Transfer Rates:

- WebSocket messages: $\leq$ 1KB per message
- Terminal data: 10KB/sec (typical), 100KB/sec (peak)
- Metrics data: 2KB per device per minute

Synchronization Mechanisms:

- Optimistic locking for database updates (version field)
- Request ID for correlating commands and responses
- Timestamp for message ordering

4 NON-FUNCTIONAL REQUIREMENTS

4.1 Performance Requirements

4.1.1 Response Time

- **API Response Time:**
 - Simple queries (GET user, GET container list): $\leq$ 100ms (95th percentile)
 - Complex queries (Dashboard statistics): $\leq$ 500ms (95th percentile)
 - Container creation API call: $\leq$ 200ms (returns PENDING immediately)
- **Container Operations:**
 - Container creation (end-to-end): 5-10 seconds (target), 15 seconds (maximum)
 - Container stop: 2-5 seconds
 - Container restart: 3-8 seconds

- Container deletion: 1-3 seconds
- **WebSocket Latency:**
 - Message delivery: ~50ms (LAN), ~200ms (WAN)
 - Terminal keystroke latency: ~100ms
 - Status update propagation: ~2 seconds

4.1.2 Throughput

- **Concurrent Users:** 500+ simultaneous users
- **API Requests:** 1000 requests/minute (backend)
- **Container Operations:** 50 containers created per minute
- **WebSocket Connections:** 500+ persistent connections

4.1.3 Scalability

- **Devices:** Support 500+ lab computers
- **Containers:** 2000+ concurrent running containers
- **Organizations:** 50+ institutions (multi-tenancy)
- **Database:** 10 million+ records (containers, logs)

4.1.4 Resource Utilization

- **Backend Memory:** ~4GB RAM under normal load
- **Backend CPU:** ~50% CPU usage with 100 devices
- **Agent Memory:** ~200MB RAM per agent
- **Database:** ~8GB RAM for 500 devices

4.1.5 Capacity

- **Concurrent Terminals:** 200+ simultaneous terminal sessions
- **User Accounts:** 10,000+ registered users
- **Historical Data:** 6 months retention for logs and metrics

4.2 Security Requirements

4.2.1 Authentication

- **User Authentication:** Username/password with BCrypt hashing (cost factor 10)
- **Session Management:** JWT tokens with 24-hour expiration
- **Agent Authentication:** Enrollment token (one-time use, 24-hour validity) + device ID
- **Token Validation:** HMAC-SHA256 signature verification
- **Password Policy:** Minimum 8 characters, require alphanumeric (future enhancement)
- **Brute Force Protection:** Rate limiting on login endpoint (5 attempts per minute)

4.2.2 Authorization

- **Role-Based Access Control (RBAC):** Three roles (STUDENT, FACULTY, ADMIN)
- **Resource Ownership:** Students can only access their own containers
- **Organization Isolation:** Users can only see resources in their organization
- **Admin Privileges:** Admins can manage all resources in their organization
- **API Authorization:** Every endpoint checks JWT claims and role

4.2.3 Data Protection

- **Encryption in Transit:** HTTPS/WSS with TLS 1.3 (production)
- **Encryption at Rest:** PostgreSQL encryption (optional, via pg_crypto)
- **Password Storage:** BCrypt hashed passwords, never stored in plaintext
- **Sensitive Data:** Enrollment tokens hashed in logs
- **SQL Injection Prevention:** Parameterized queries (JPA/Hibernate)
- **XSS Prevention:** Input sanitization, Content-Security-Policy header

4.2.4 Container Security

- **Isolation:** Each container runs in isolated namespace
- **No Privileged Access:** Containers run without `--privileged` flag
- **Resource Limits:** CPU, RAM, disk quotas enforced by Docker
- **Network Isolation:** Bridge network mode, no inter-container communication
- **Image Security:** Use official Docker Hub images, vulnerability scanning (future)

4.2.5 Privacy

- **Data Residency:** All data stored on-premise (no third-party cloud)
- **User Activity:** Logged for audit purposes, not shared externally
- **Container Content:** Not inspected or monitored by system
- **GDPR Compliance:** User data deletion on request (future enhancement)

4.2.6 Verification and Validation

- **Input Validation:** Bean Validation (JSR-380) on all API inputs
- **JWT Verification:** Signature and expiration checks on every request
- **Enrollment Token Verification:** Check expiration, organization, used status
- **Device Heartbeat:** Mark devices OFFLINE if heartbeat missing for 60 seconds
- **Container State Validation:** State machine prevents invalid transitions

4.3 Software Quality Attributes

4.3.1 Adaptability

The system is designed for easy adaptation to different environments:

- Configuration externalized in application.properties and config.yaml
- Support for multiple organizations without code changes
- Docker image selection configurable per organization
- Quota policies adjustable via admin interface
- Database schema supports multi-tenancy

4.3.2 Availability

- **Target Uptime:** 99%+ (8.76 hours downtime per year)
- **Agent Reconnection:** Automatic reconnection on network failure (5-second retry)
- **Database:** Primary-replica setup for high availability (future)
- **Graceful Degradation:** System continues with reduced capacity if devices go offline
- **Maintenance Window:** Scheduled maintenance during low-usage hours (2-4 AM)

4.3.3 Correctness

- **Functional Correctness:** All features tested against requirements
- **Data Integrity:** Database constraints prevent invalid states
- **Resource Accounting:** Accurate tracking of allocated vs available resources
- **State Consistency:** Container states synchronized between backend and Docker

4.3.4 Flexibility

- **Pluggable Scheduling:** Scoring weights configurable for different algorithms
- **Image Support:** Easy to add new Docker images
- **Authentication:** JWT-based, can integrate with LDAP/OAuth2 (future)
- **Database:** ORM layer allows database migration (PostgreSQL to others)

4.3.5 Interoperability

- **REST API:** Standard HTTP/JSON interface for integration
- **WebSocket:** Standard RFC 6455 protocol
- **Docker:** Compatible with any Docker Engine 24+
- **Browser Compatibility:** Works with all modern browsers
- **Cross-Platform:** Backend runs on Windows/Linux/macOS

4.3.6 Maintainability

- **Code Documentation:** Javadoc for Java, docstrings for Python, JSDoc for JavaScript

- **Logging:** Structured logs (SLF4J in Java, logging in Python)
- **Error Handling:** Centralized exception handlers
- **Modularity:** Layered architecture (Controller, Service, Repository)
- **Version Control:** Git with semantic versioning

4.3.7 Portability

- **Backend:** Java-based, runs on any JVM-compatible OS
- **Agent:** Python-based, supports Windows 10/11 and Linux (Ubuntu 20.04+)
- **Frontend:** Web-based, no installation required
- **Database:** PostgreSQL available on all major platforms
- **Containerization:** Backend can be dockerized for easy deployment

4.3.8 Reliability

- **Mean Time Between Failures (MTBF):** 720 hours (30 days)
- **Mean Time To Recovery (MTTR):** <30 minutes
- **Data Backup:** Daily database backups with 30-day retention
- **Fault Tolerance:** Agent failures don't affect other agents
- **Transaction Management:** Database transactions ensure consistency

4.3.9 Reusability

- **Component Reuse:** Service layer can be used by multiple controllers
- **Code Libraries:** Utility classes for common operations
- **Template Containers:** Pre-configured images for common use cases
- **API:** RESTful design allows third-party integrations

4.3.10 Robustness

- **Error Handling:** Try-catch blocks with meaningful error messages
- **Input Validation:** Reject invalid inputs before processing
- **Resource Limits:** Prevent resource exhaustion (memory, disk, connections)
- **Timeout Handling:** WebSocket and HTTP timeouts prevent hanging
- **Graceful Shutdown:** Clean closure of connections and services

4.3.11 Testability

- **Unit Tests:** JUnit for Java, pytest for Python, Jest for JavaScript
- **Integration Tests:** Spring Boot Test framework
- **API Tests:** Postman collections for endpoint testing
- **Mocking:** Mockito for mocking dependencies
- **Test Coverage:** Target 70%+ code coverage

4.3.12 Usability

- **Ease of Learning:** Students can create first container in <5 minutes
- **User Documentation:** Student guide, admin guide, installation manual
- **Error Messages:** Clear, actionable error descriptions
- **UI Consistency:** Material Design throughout
- **Accessibility:** Keyboard navigation, screen reader support (WCAG 2.1 Level A)

5 OTHER REQUIREMENTS

5.1 Legal and Licensing

- System complies with Docker Desktop licensing terms
- Open-source dependencies comply with their respective licenses (Apache 2.0, MIT)
- No third-party proprietary code used
- Institutional approval required before deployment

5.2 Operational Requirements

- Backend server must be accessible 24/7
- Lab computers should be powered on during usage hours
- Regular database backups (daily recommended)
- Log rotation to prevent disk space exhaustion
- Monitoring and alerting setup (future enhancement)

5.3 Environmental Requirements

- Air-conditioned server room for backend server
- UPS (Uninterruptible Power Supply) for backend and network equipment
- Reliable network infrastructure (switches, routers)
- Firewall configuration allowing port 8081 (WebSocket)

5.4 Disaster Recovery

- Database backup and restore procedures documented
- Configuration files version-controlled in Git
- Container data not persisted (ephemeral by design)
- Recovery Time Objective (RTO): 4 hours
- Recovery Point Objective (RPO): 24 hours (daily backups)

5.5 Training Requirements

- Administrator training: 4-hour session covering installation, configuration, troubleshooting

- Faculty training: 1-hour session on monitoring and reservations
- Student onboarding: 15-minute video tutorial or written guide
- Documentation: Admin guide, student guide, API reference

5.6 Support and Maintenance

- Bug fixes: Within 48 hours for critical issues
- Feature enhancements: Quarterly releases
- Security patches: Within 24 hours of discovery
- Documentation updates: Concurrent with code changes

APPENDIX A: GLOSSARY

Agent: Software component installed on lab computers that manages Docker containers and communicates with the backend via WebSocket.

API (Application Programming Interface): Set of HTTP endpoints for programmatic access to system functionality.

Backend: Central server application (Spring Boot) that coordinates container orchestration, user management, and agent communication.

BCrypt: Password hashing algorithm used for secure password storage.

Container: Isolated, lightweight virtualized environment created using Docker technology.

Device: Lab computer with agent installed that can run containers.

Docker: Platform for developing, shipping, and running applications in containers.

Enrollment Token: One-time-use authentication token for registering new agent devices.

Frontend: Web-based user interface (React application) for students, faculty, and administrators.

Heartbeat: Periodic signal sent by agent to backend indicating it is still online (30-second interval).

JWT (JSON Web Token): Compact token format for securely transmitting information between parties.

Lab: Physical computer lab location containing multiple devices.

Metrics: System resource measurements (CPU, RAM, disk usage) collected by agents.

Organization: Top-level entity representing an institution (university, college) in multi-tenant architecture.

PostgreSQL: Open-source relational database management system used for persistent storage.

Quota: Resource limit (CPU cores, RAM) assigned to a user or organization.

RBAC (Role-Based Access Control): Authorization model where permissions are assigned based on user roles.

REST (Representational State Transfer): Architectural style for designing networked applications using HTTP.

Scheduler: Component that selects the best device for container allocation using scoring algorithm.

Terminal: Web-based command-line interface for interacting with containers (xterm.js).

WebSocket: Protocol providing full-duplex communication channels over TCP connection.

WSS (WebSocket Secure): Encrypted WebSocket connection using TLS.

APPENDIX B: ANALYSIS MODEL

B.1 Resource Utilization Model

Formula for Lab Utilization:

Utilization (%) = (Allocated CPU / Total CPU) * 100

Example:

Lab: 50 computers, each with 8 CPU cores = 400 total cores

Current: 30 containers, each using 2 cores = 60 allocated cores

Utilization: $(60 / 400) * 100 = 15\%$

B.2 Cost Savings Model

AWS Cost for 200 Students:

Instance: t3.medium (2 vCPU, 4GB RAM)

Cost: \$0.0416/hour = \$30/month per instance

200 students * \$30 = \$6,000/month = \$72,000/year

INR equivalent: Rs. 72,000 * 83 = Rs. 59,76,000/year

Rounded: Rs. 60 lakhs/year

CampusCompute Cost:

Backend Server: Rs. 50,000/year (VPS) or Rs. 1,50,000 (on-premise, one-time)

Internet: Rs. 30,000/year

Total: Rs. 80,000/year (cloud) or Rs. 1,80,000 first year (on-premise)

Savings:

AWS: Rs. 60,00,000/year
CampusCompute: Rs. 80,000/year
Savings: Rs. 59,20,000/year (98.7%)

B.3 Device Scoring Model

Mathematical Formula:

Score = Sum($w_i * s_i$) where $i = 1$ to 5

s_1 = CPU availability score = (available_cpu / total_cpu) * 100
 s_2 = RAM availability score = (available_ram / total_ram) * 100
 s_3 = Disk availability score = (available_disk / total_disk) * 100
 s_4 = Container load score = (1 - active_containers / max_containers) * 100
 s_5 = Health score = {100 if healthy, 50 if warning, 0 if error}

Weights: $w_1=0.30$, $w_2=0.30$, $w_3=0.15$, $w_4=0.15$, $w_5=0.10$
Sum(w_i) = 1.0 (normalized)

B.4 Capacity Planning Model

Formula:

Max Concurrent Containers = Sum(Device CPU Cores / Avg Container CPU)

Example:

50 devices * 8 cores = 400 total cores
Average container: 2 cores
Max containers: 400 / 2 = 200 containers
Safety margin (80%): 200 * 0.8 = 160 containers

APPENDIX C: ISSUES LIST

Open Issues (as of October 2026):

1. Issue #1: Terminal Disconnection

- *Description:* WebSocket terminal disconnects after 30 minutes of inactivity
- *Priority:* Medium
- *Status:* Under investigation
- *Proposed Solution:* Implement keepalive pings every 60 seconds

2. Issue #2: Docker Installation Requires Restart

- *Description:* Agent installer requires system restart after Docker installation

- *Priority:* Low
- *Status:* By design (Docker requirement)
- *Resolution:* Document in user guide, notify user during installation

3. **Issue #3: Multi-Device Testing Incomplete**

- *Description:* System tested on single device, needs validation with 10-20 devices
- *Priority:* High
- *Status:* Planned for next phase
- *Timeline:* 2-3 weeks

4. **Issue #4: Linux Agent Installer Not Available**

- *Description:* Automated installer only for Windows, Linux requires manual setup
- *Priority:* Medium
- *Status:* Future enhancement
- *Proposed Solution:* Create shell script installer for Linux

5. **Issue #5: GPU Support Missing**

- *Description:* Containers cannot access GPU for ML/AI workloads
- *Priority:* Low (enhancement)
- *Status:* Future roadmap
- *Requirements:* Docker with NVIDIA GPU support, CUDA toolkit

Resolved Issues:

1. **Issue #R1: LazyInitializationException (Resolved Oct 5, 2026)**

- *Solution:* Added JOIN FETCH queries and Jackson Hibernate6Module

2. **Issue #R2: Database Constraint Error on RESTARTING Status (Resolved Oct 5, 2026)**

- *Solution:* Updated database constraint to include STOPPING, RESTARTING states

3. **Issue #R3: Container Stop Shows Error on First Click (Resolved Oct 5, 2026)**

- *Solution:* Implemented AWS-style intermediate states with delayed refresh